Prueba Técnica

Stack: .NET 8 Web API · Arquitectura Hexagonal · DDD táctico · CQRS + Mediator · Repository + Unit of Work · Specification · Eventos con RabbitMQ · Frontal en React o Angular

DEDICACIÓN ESTIMADA	VARIANTE DE FRONTAL
6–8 h de trabajo efectivo	☐ React ☐ Angular

La variante de frontal la indicamos al enviarte la prueba. Si no viene marcada, elige tú y justifícalo en el README.

1. Objetivo

Construir un micro-MVP de **gestión de reservas de recursos** (salas, puestos, vehículos…) con un backend .NET 8 desacoplado y un frontal SPA. Queremos ver cómo aplicas **arquitectura hexagonal**, **DDD táctico**, **CQRS**, **comunicación asíncrona por eventos** y **componentización** en el frontal.

No buscamos una aplicación completa ni una interfaz pulida: valoramos más un núcleo bien diseñado, probado y explicado que muchas funcionalidades a medias. Si no llegas a todo, prioriza en este orden: **dominio → casos de uso → eventos → API → frontal**, y deja constancia en el README de lo que falta.

2. Alcance funcional

ID	Funcionalidad
F1	**Crear reserva** en estado `Pending` con `ResourceId`, `Customer { Name, Email }`, `Start`, `End`.
F2	**Listar reservas** con filtro por `ResourceId` y rango de fechas (`from`, `to`), **paginación y ordenación en servidor**.
F3	**Confirmar** una reserva `Pending`. Debe bloquear solapes con otras `Confirmed` del mismo `ResourceId`.
F4	**Cancelar** una reserva `Pending` o `Confirmed`, con motivo opcional.
F5	**Ver detalle** de una reserva en una ruta dedicada `/bookings/:id`.
F6	**Notificación asíncrona**: al confirmar o cancelar se publica un evento de integración en RabbitMQ. Un servicio consumidor independiente lo procesa y registra una notificación simulada (log estructurado + persistencia).

Fuera de alcance: autenticación/autorización, i18n, despliegue en cloud, diseño visual cuidado y envío real de emails.

3. Reglas de dominio

- **Estados:** `Pending`, `Confirmed`, `Cancelled` (terminal).
- **Transiciones válidas:** `Pending → Confirmed | Cancelled`, `Confirmed → Cancelled`. Cualquier otra transición es un error de dominio (`booking.invalid_transition`).
- **Fechas:** `Start < End` y `Start` en el futuro respecto al reloj del sistema. El reloj se inyecta (`TimeProvider` de .NET 8 o un puerto `IClock`); nada de `DateTime.Now` en el dominio.
- **Zona horaria:** la API recibe y devuelve ISO 8601 con offset (`DateTimeOffset`) y persiste en UTC.
- **Solape:** no puede haber dos reservas `Confirmed` solapadas del mismo `ResourceId`. Los intervalos son **semiabiertos** `[Start, End)`: 10:00–11:00 y 11:00–12:00 **no** se solapan. Las `Pending` sí pueden solaparse entre sí.

- **Cliente:** `Name` obligatorio (máx. 100 caracteres) y `Email` con formato válido.

4. Arquitectura y estructura de la solución

Arquitectura hexagonal (puertos y adaptadores) con las dependencias apuntando siempre hacia el dominio. Estructura orientativa:

```
src/
  Bookings.Domain/                 Agregado, Value Objects, eventos, Specifications
  Bookings.Application/            Commands, Queries, handlers, puertos, behaviors
  Bookings.Infrastructure/         EF Core, repositorios, UoW, Outbox, publicación en RabbitMQ
  Bookings.Api/                    Endpoints REST, ProblemDetails, OpenAPI, composition root
  Bookings.Contracts/              Eventos de integración (contrato público y versionado)
  Bookings.Notifications.Worker/   Consumidor de RabbitMQ (proceso independiente)
web/                               SPA en React o Angular
tests/
  Bookings.Domain.Tests/
  Bookings.Application.Tests/
  Bookings.Api.IntegrationTests/
docker-compose.yml                 PostgreSQL + RabbitMQ (management) [+ api, worker, web]
```

Reglas de dependencia: `Domain` no referencia ningún otro proyecto ni paquete de infraestructura. `Application` solo depende de `Domain`. `Infrastructure` implementa los puertos de `Application`. `Api` solo orquesta y registra dependencias. El `Worker` depende únicamente de `Contracts` (no del dominio de reservas).

Persistencia: PostgreSQL con EF Core (recomendado, levantado en `docker-compose`). SQLite es aceptable si lo justificas, teniendo en cuenta las implicaciones de concurrencia (ver 5.4).

5. Patrones obligatorios

5.1 DDD táctico: agregado y Value Objects OBLIGATORIO

- `Booking` es el **Aggregate Root**. Su estado solo cambia a través de métodos con intención de negocio (`Booking.Create(...)`, `Confirm(...)`, `Cancel(reason)`). Sin setters públicos ni modelo anémico.
- **Value Objects** inmutables con igualdad por valor y auto-validación: `BookingId`, `ResourceId`, `Email`, `Customer` y `DateRange` (con `Overlaps(DateRange other)`).
- Las reglas de negocio viven en el dominio, no en los handlers ni en los controladores.
- Errores de dominio tipados y con código estable (p. ej. `booking.overlap`, `booking.invalid_transition`, `booking.start_in_past`). Puedes usar excepciones de dominio o un tipo Result: elige y justifica.

5.2 Specification OBLIGATORIO

- La regla de solape se expresa como una Specification reutilizable, p. ej. `ConfirmedOverlappingBookingSpec(resourceId, range, excludeId)`.
- Debe poder **evaluarse en base de datos** (traducible a `Expression<Func<Booking,bool>>` / consulta EF): no se acepta cargar todas las reservas en memoria para comprobar solapes.
- Los filtros del listado (F2) también deben modelarse como Specification o como un objeto de criterios equivalente.

5.3 CQRS + Mediator OBLIGATORIO

- **Commands:** `CreateBookingCommand`, `ConfirmBookingCommand`, `CancelBookingCommand`. Un handler por caso de uso.

- **Queries:** `GetBookingsQuery` (paginada) y `GetBookingByIdQuery`. Devuelven DTOs/read models, nunca entidades de dominio. Las queries pueden saltarse el repositorio y leer directamente con `AsNoTracking()` o proyecciones.
- **Mediator:** MediatR, *Mediator* (source generator) o una implementación propia. Justifica la elección (licencia, rendimiento, simplicidad).
- **Pipeline behaviors** obligatorios: `ValidationBehavior` (FluentValidation sobre los commands) y `LoggingBehavior`. Un `TransactionBehavior` es opcional si el commit ya está centralizado.
- Los endpoints de la API son finos: mapean la petición a un command/query, lo envían al mediator y traducen la respuesta.

5.4 Repository + Unit of Work OBLIGATORIO

- Puertos en `Application` (o `Domain`): `IBookingRepository` orientado al agregado (`GetByIdAsync`, `AddAsync`, `AnyAsync(spec)`) e `IUnitOfWork.SaveChangesAsync`.
- Adaptadores con EF Core en `Infrastructure`. **No** se acepta un repositorio genérico que exponga `IQueryable` hacia fuera.
- **Un único commit por command**, que incluye el agregado y los mensajes de la Outbox (ver 6.3).
- **Concurrencia:** dos confirmaciones simultáneas de reservas solapadas no pueden acabar ambas en `Confirmed`. Implementa una protección (concurrencia optimista con `RowVersion` / `xmin`, bloqueo por recurso, aislamiento serializable, restricción de exclusión en PostgreSQL...) y explícala en un ADR.

6. Eventos y mensajería (RabbitMQ)

6.1 Eventos de dominio OBLIGATORIO

- El agregado registra `BookingCreated`, `BookingConfirmed` y `BookingCancelled` en una colección interna de eventos.
- El despacho lo hace la infraestructura (p. ej. interceptor de `SaveChanges` o UoW), no el handler a mano. Los handlers de eventos de dominio viven en `Application`.

6.2 Eventos de integración OBLIGATORIO

- Definidos en `Bookings.Contracts`, separados de los eventos de dominio: `BookingConfirmedV1` y `BookingCancelledV1`.
- Campos mínimos: `EventId`, `OccurredAtUtc`, `CorrelationId`, `BookingId`, `ResourceId`, `Start`, `End`, `CustomerEmail` (y `Reason` en la cancelación).
- Un handler de evento de dominio traduce el evento de dominio a evento de integración. El dominio nunca conoce el broker.

6.3 Publicación fiable: Transactional Outbox OBLIGATORIO

- Los eventos de integración se guardan en una tabla `OutboxMessages` **en la misma transacción** que el cambio del agregado. Nada de publicar en RabbitMQ dentro del handler (doble escritura).
- Un proceso en segundo plano publica los pendientes, los marca como procesados y reintenta ante fallos.
- Puedes implementarla tú o usar la Outbox de EF Core de MassTransit. En ambos casos, explica el funcionamiento en el README.

6.4 Consumidor: Notifications.Worker OBLIGATORIO

- Proceso independiente (`Worker Service`) que consume `BookingConfirmedV1` y `BookingCancelledV1`.
- Registra una notificación simulada: log estructurado y una fila en su propia tabla/esquema `Notifications`.

- **Idempotente:** si el mismo mensaje llega dos veces, solo se registra una notificación (Inbox / deduplicación por `EventId`).
- **Resiliente:** política de reintentos configurada y cola de error / DLQ para los mensajes que fallen de forma persistente.
- Cliente de RabbitMQ libre: MassTransit (recomendado) o `RabbitMQ.Client`. Documenta la topología (exchanges, colas, bindings) en el README.

> **Flujo esperado al confirmar:** `POST /confirm` → `ConfirmBookingCommand` → `Booking.Confirm()` → `BookingConfirmed` (dominio) → `BookingConfirmedV1` en Outbox → commit único → publicador de Outbox → RabbitMQ → `Notifications.Worker` → notificación registrada.

7. API REST

Endpoint	Respuestas
`POST /api/bookings`	`201` + cabecera `Location` · `400` validación
`GET /api/bookings?resourceId=&from=&to=&page=&pageSize=&sort=`	`200` con `{ items, page, pageSize, totalCount }`
`GET /api/bookings/{id}`	`200` · `404`
`POST /api/bookings/{id}/confirm`	`200` · `404` · `409` solape · `422` transición inválida
`POST /api/bookings/{id}/cancel`	`200` · `404` · `422` transición inválida

- Todos los errores siguen **ProblemDetails (RFC 7807)** con la extensión `code` (código de error de dominio) y, en validación, el diccionario `errors` por campo.
- OpenAPI/Swagger disponible, CORS configurado para el frontal, health checks en `/health` (base de datos y RabbitMQ).
- Propagación de `CorrelationId` (cabecera `X-Correlation-Id`) desde la API hasta los mensajes y logs del Worker.

8. Frontal: React o Angular

8.1 Requisitos comunes

- TypeScript en modo `strict`. URL base de la API configurable por entorno.
- Rutas: `/bookings` (listado + filtros + paginación), `/bookings/new` (alta) y `/bookings/:id` (detalle).
- **Capa de acceso a datos separada** de los componentes (cliente/servicio HTTP con DTOs tipados). Ningún componente llama a `fetch` / `HttpClient` directamente.
- Gestión centralizada de errores: los ProblemDetails se muestran de forma legible (mensaje por `code` y errores por campo en el formulario).
- Estados de carga, vacío y error en todas las vistas. Los filtros del listado se reflejan en la URL (query params).
- No se requiere librería de estado global (Redux, NgRx...). Si la usas, justifícalo.

8.2 Componentes solicitados

1. Tabla reutilizable y genérica `GenericTable<T>`. No conoce el dominio de reservas, sus filas se pueden activar con teclado (`Enter`) y tiene tests propios.

React

```
type GenericTableProps<T> = {
  items: T[];
  header: ReactNode;
  renderRow: (item: T) => ReactNode;
  onRowClick?: (item: T) => void;
  getRowKey: (item: T) => string;
  emptyText?: string; // "Sin datos"
  loading?: boolean;
};
```

Angular

```
@Component({ selector: 'app-generic-table',
             standalone: true, ... })
class GenericTableComponent<T> {
  items = input.required<T[]>();
  emptyText = input('Sin datos');
  loading = input(false);
  rowClick = output<T>();
  // <ng-template #header> y
  // <ng-template #row let-item> vía contentChild
}
```

2. Vista detalle `BookingDetails` en `/bookings/:id`: muestra recurso, cliente, rango `Start–End` en hora local, estado (badge) y timestamps (`createdAt`, `confirmedAt`, `cancelledAt`). Acciones según estado: *Confirmar* si `Pending`; *Cancelar* si `Pending` o `Confirmed`. Si la acción falla, se muestra el error y el estado mostrado no cambia.

3. Formulario de alta `BookingForm`: validación en cliente equivalente a la del servidor (`Start < End`, fecha futura, email), aunque el servidor sigue siendo la fuente de verdad: sus errores se pintan por campo.

8.3 Stack por variante

	React	Angular
Versión / tooling	React 18+ con Vite	Angular 17+ con standalone components
Routing	React Router	Angular Router
Datos	TanStack Query (recomendado) o hooks propios	`HttpClient` + interceptor de errores; signals o RxJS
Formularios	React Hook Form + Zod (recomendado)	Reactive Forms tipados
Tests	Vitest + Testing Library	Jest o Karma + TestBed / Angular Testing Library

9. Testing (obligatorio)

- **Dominio:** transiciones válidas e inválidas, `DateRange` y casos límite de solape (incluida la adyacencia).
- **Aplicación:** `ConfirmBookingHandler` con y sin solape, y `ValidationBehavior`.
- **Integración:** al menos un test con `WebApplicationFactory` que confirme una reserva y verifique que se publica `BookingConfirmedV1` (Testcontainers con PostgreSQL/RabbitMQ o el test harness de MassTransit).
- **Frontal:** tests de `GenericTable` (cabecera + filas, texto vacío, click/Enter emite el item).

No medimos cobertura: valoramos que los tests prueben comportamiento relevante y sean legibles.

10. Criterios de aceptación

ID	Escenario
CA1	**Crear:** con datos válidos a futuro se devuelve `201`, la reserva queda `Pending` y aparece en el listado.
CA2	**Confirmar sin solape:** pasa a `Confirmed`, se publica `BookingConfirmedV1` y el Worker registra la notificación.
CA3	**Bloqueo de solape:** con una `Confirmed` 10:00–11:00 en `Room-A`, confirmar otra 10:30–11:30 en `Room-A` devuelve `409` con `code = booking.overlap`, no cambia el estado y **no se publica ningún evento**.
CA4	**Adyacencia:** con la anterior confirmada, una reserva 11:00–12:00 en `Room-A` se confirma sin problema.
CA5	**Transición inválida:** cancelar una reserva ya `Cancelled` devuelve `422` con `code = booking.invalid_transition`.
CA6	**Validación:** `Start` en el pasado o `Start ≥ End` devuelve `400` con errores por campo, que el formulario muestra junto a cada input.
CA7	**Tabla:** renderiza cabecera y filas; sin datos muestra `emptyText`; click o `Enter` en una fila navega al detalle.
CA8	**Detalle:** `/bookings/:id` muestra los datos; si confirmas desde el detalle y hay solape, se muestra el error y el estado no cambia.
CA9	**Filtros y paginación:** filtrar por recurso y rango devuelve solo las reservas que intersectan; los filtros persisten en la URL al recargar.
CA10	**Broker caído:** con RabbitMQ parado, confirmar devuelve `200`; al levantarlo, el evento se publica y el Worker lo procesa.
CA11	**Idempotencia:** si el mismo mensaje se entrega dos veces, el Worker registra una única notificación.
CA12	**Arranque:** siguiendo el README en una máquina limpia (Docker + .NET 8 SDK + Node LTS), el sistema levanta y funciona.

11. Entregables

1. **Repositorio Git** (GitHub/GitLab con acceso para nosotros), con **commits incrementales**: valoramos ver la evolución, no un único commit final.
2. `docker-compose.yml` con PostgreSQL y RabbitMQ (con consola de management). Incluir API, Worker y web es un plus.
3. **README** (5–10 min de lectura) con:
   - Cómo ejecutar (idealmente un solo comando), datos semilla y cómo lanzar los tests.
   - Diagrama de arquitectura y diagrama de secuencia del flujo de eventos (Mermaid).
   - Topología de RabbitMQ (exchanges, colas, DLQ).
   - Variante de frontal elegida y estructura de carpetas.
4. **5 mini-ADR** (contexto, decisión, alternativas, consecuencias; máx. ½ página cada uno):
   a. Validación: FluentValidation y dónde se valida cada cosa (VO, command, frontal).
   b. Elección del mediator.
   c. Modelado del solape y estrategia de concurrencia.
   d. Outbox, broker e idempotencia del consumidor.
   e. Diseño de la tabla genérica en el framework elegido.

5. **Sección final** “Qué haría con más tiempo” y declaración de **uso de herramientas de IA** (cuáles y para qué). Su uso está permitido; lo que valoramos es que entiendas y puedas defender cada decisión.

12. Bonus (opcional)

- BONUS Actualización en tiempo real del listado y del detalle (SignalR) cuando cambia el estado de una reserva.
- BONUS Tests de arquitectura (NetArchTest / ArchUnitNET) que verifiquen las reglas de dependencia.
- BONUS Trazas distribuidas con OpenTelemetry de API → RabbitMQ → Worker.
- BONUS Test de concurrencia que demuestre que dos confirmaciones simultáneas solapadas no terminan ambas en `Confirmed`.
- BONUS Pipeline de CI (GitHub Actions / GitLab CI) con build y tests, y un test e2e con Playwright.

13. Criterios de evaluación

Área	Peso	Qué miramos
Dominio y arquitectura	25 %	Agregado rico, VOs, reglas en el dominio, dependencias correctas entre capas.
Patrones de aplicación	20 %	CQRS limpio, behaviors, Repository/UoW bien acotados, Specification evaluable en BD, concurrencia.
Eventos y mensajería	20 %	Separación dominio/integración, Outbox, idempotencia, reintentos/DLQ, correlación.
Frontal	15 %	Componentización real, tabla genérica, separación de la capa de datos, manejo de errores y estados.
Testing	10 %	Tests relevantes y legibles en todas las capas solicitadas.
Documentación y DX	10 %	README claro, ADRs razonados, arranque sencillo, historial de commits.

> **Entrevista de revisión:** tras la entrega haremos una sesión de 45–60 min en la que nos explicarás la solución, defenderás las decisiones de los ADR y haremos alguna pequeña modificación en vivo sobre tu código.